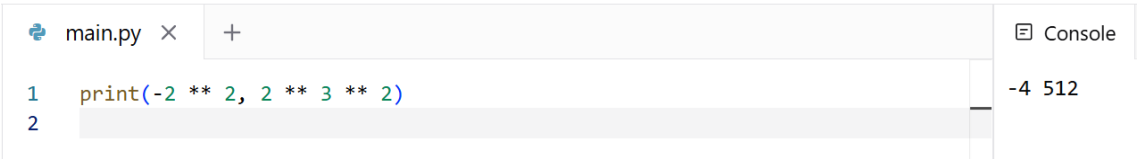
```
print(-2 ** 2, 2 ** 3 ** 2)
```

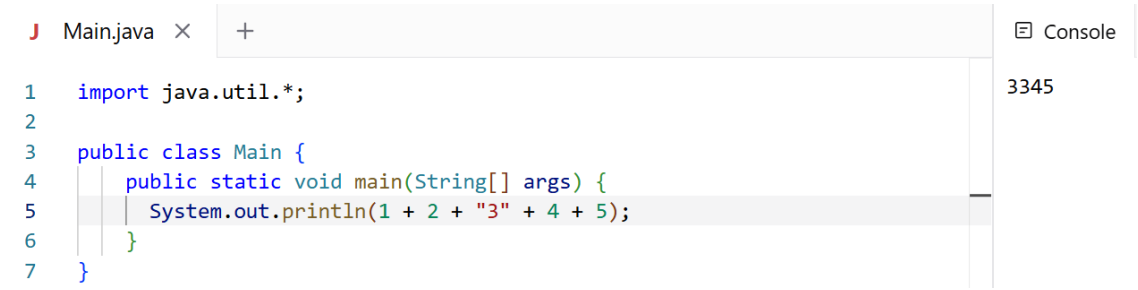
Palpite: não roda, Saída -4, 512




```
System.out.println(1 + 2 + "3" + 4 + 5);
```

Palpite: aqui temos várias adições no mesmo trecho, acredito que por conta da string “3” no meio da operação, ou o código irá somar 1 + 2 e 4 + 5 e deixar o “3” no meio, printando 339, ou não irá compilar.

Resultado: o resultado que temos é 3345, acredito que, no processo de implementar as operações, os dois primeiros elementos de fato são números inteiros, já o terceiro é uma string, então ele entenderá que os outros dois últimos também serão strings, então ao invés de somar ele concatena os números, temos 1 + 2 = 3, 3 + “3” que forma outra string “33”, depois temos “33” + 4 = “334”, e por fim temos “334”+5 = “3345”. Portanto temos um erro de execução.



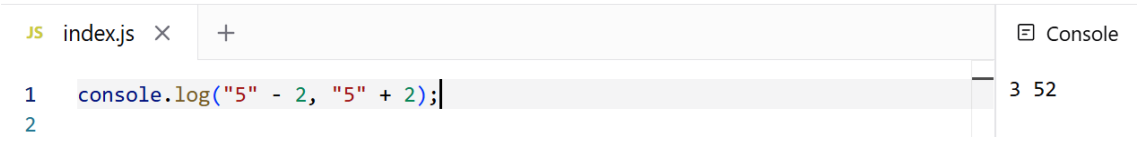

```
console.log("5" - 2, "5" + 2);
```

Não temos erro js é ruim mesmo.

O resultado impresso no console será o número 3 seguido do texto "52". Isso acontece por causa de um comportamento do JavaScript chamado coerção implícita de tipo, onde a linguagem tenta adivinhar sua intenção e converte os valores automaticamente dependendo do operador utilizado na expressão.

Na primeira parte, ao tentar subtrair o número 2 do texto "5", a linguagem identifica que o operador de subtração serve exclusivamente para contas matemáticas. Para não quebrar o código, o JavaScript converte silenciosamente a string "5" no número inteiro 5, realiza a conta tradicional e devolve o resultado 3. O mesmo aconteceria com qualquer outro operador estritamente matemático, como a multiplicação ou a divisão, onde as strings sempre seriam forçadas a virar números.

Já na segunda parte da expressão, a regra muda porque o operador de adição possui uma função dupla: ele serve tanto para somar números quanto para concatenar e juntar textos. No JavaScript, a regra determina que se houver uma string de qualquer lado do operador de adição, a operação de juntar textos ganha a prioridade absoluta. Assim, em vez de transformar o "5" em número, a linguagem transforma o número 2 em texto e simplesmente une as duas partes, gerando a string final "52".

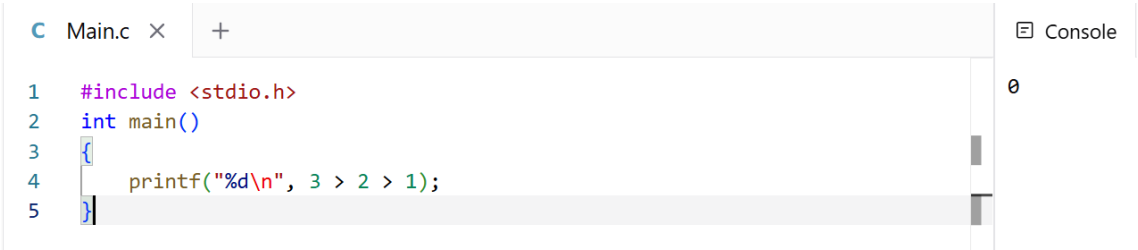



```
printf("%d\n", 3 > 2 > 1);
```

Em C, operadores relacionais como > são avaliados da esquerda para a direita. A expressão 3 > 2 > 1 é processada pelo compilador como (3 > 2) > 1.

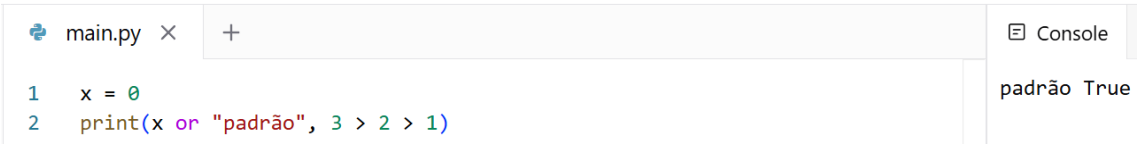
1. **3 > 2** é avaliado primeiro. Como 3 é maior que 2, esta afirmação é verdadeira. Na linguagem C, uma expressão relacional verdadeira resulta no número inteiro **1**.
2. A expressão, portanto, torna-se **1 > 1**.
3. Como 1 não é maior que 1, esta afirmação é falsa, o que resulta no número inteiro **0**.

A função printf imprime esse resultado inteiro final.



```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

Ele irá retornar “padrão” e True, pois quando olhamos para as partes do print temos duas operações, a primeira x or “padrao”,o operador or retorna valores que são Truly, ou seja, valores verdadeiros e x é 0 ou seja, False. Já a segunda operação 3 > 2 > 1 temos um and implícito que transforma a operação em (3 >2)and (2>1) ambos os resultados da operação são True.




```
console.log(0 == "", "" == "0", 0 == "0");
```

O resultado executado por esse código em JavaScript é **true false true**, e isso ocorre devido à forma como a linguagem lida com a conversão automática de tipos (chamada de coerção) ao usar o operador de igualdade solta, representado por dois sinais de igual (==).

Na primeira comparação (0 == ""), temos um número sendo comparado a uma string (texto) vazia. Quando tipos diferentes se encontram nesse tipo de igualdade, o JavaScript tenta convertê-los para o mesmo formato. A regra interna da linguagem define que uma string vazia, quando forçada a se tornar um número, vale zero. Com isso, a linguagem interpreta a expressão como sendo zero igual a zero, o que resulta em verdadeiro (true).

Já na segunda comparação ("" == "0"), o comportamento muda. Como ambos os lados já são do tipo string, nenhuma conversão matemática acontece. O motor do JavaScript apenas avalia de forma literal o conteúdo dos textos e percebe que uma string totalmente sem caracteres é evidentemente diferente de uma string que contém o caractere "0", retornando assim o valor falso (false).

Na terceira e última comparação (0 == "0"), a regra de conversão entra em cena novamente, pois temos o encontro de um número literal com uma string. A string que contém o texto "0" é transformada no número matemático zero para que a igualdade possa ser testada. Com a conversão feita, a expressão passa a ser lida novamente como zero igual a zero, o que resulta em verdadeiro (true).

index.js

```
1  console.log(0 == "", "" == "0", 0 == "0");
2
```

Console I/O

```
true false true
```